

Problem Title

Sieve of Eratosthenes – Generate Prime Numbers

Difficulty Level: Medium(Efficient Version)

Problem Description (Including Scenario)

This problem was asked by **Square**.

Prime numbers play a crucial role in **cryptography, mathematics, and computer science**. One of the most efficient ways to generate all prime numbers **less than a given number N** is the **Sieve of Eratosthenes** algorithm.

How the algorithm works:

1. Assume all numbers from 2 to $N-1$ are prime.
2. Start with the first prime number 2.
3. Mark all multiples of 2 as **composite**.
4. Move to the next unmarked number and repeat.
5. Continue until all primes less than N are processed.
6. All **unmarked numbers** are prime.

Visualized Example (Explanation)

Enter Number:

Generate Primes

0	1	2	3	4	5	6	7	8	9	10	11	12
13	14	15	16	17	18	19	20	21	22	23	24	25
26	27	28	29	30	31	32	33	34	35	36	37	38
39	40	41	42	43	44	45	46	47	48	49	50	51
52	53	54	55	56	57	58	59	60	61	62	63	64
65	66	67	68	69	70	71	72	73	74	75	76	77
78	79	80	81	82	83	84	85	86	87	88	89	90
91	92	93	94	95	96	97	98	99	100			



GLA
UNIVERSITY
Recognised by UGC Under Section 2(f) & 12B Status

Accredited with **A+** Grade by **NAAC**
Mathura | Greater Noida

Department of Computer Engineering and Application
Institute of Engineering and Technology

1	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49	50
51	52	53	54	55	56	57	58	59	60
61	62	63	64	65	66	67	68	69	70
71	72	73	74	75	76	77	78	79	80
81	82	83	84	85	86	87	88	89	90
91	92	93	94	95	96	97	98	99	100

N

Sieve of Eratosthenes (10×10)

	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49	50
51	52	53	54	55	56	57	58	59	60
61	62	63	64	65	66	67	68	69	70
71	72	73	74	75	76	77	78	79	80
81	82	83	84	85	86	87	88	89	90
91	92	93	94	95	96	97	98	99	100

Primes:

2, 3, 5, 7,
 11, 13, 17,
 19, 23, 29,
 31, 37, 41,
 43, 47, 53,
 59, 61, 67,
 71, 73, 79,
 83, 89, 97



Example

$N = 20$

Step-by-step marking:

2 → mark multiples: 4, 6, 8, 10, 12, 14, 16, 18

3 → mark multiples: 6, 9, 12, 15, 18

5 → mark multiples: 10, 15

Unmarked (Prime Numbers):

2, 3, 5, 7, 11, 13, 17, 19

Input Format

- A single integer N
- Generate all prime numbers **less than N**

Output Format

- A list of all prime numbers less than N

Constraints

$$2 \leq N \leq 10^6$$

Topic Covered

- Number Theory
- Prime Numbers
- Arrays
- Mathematical Optimization

Approaches

1. Brute Force Approach (Not Recommended)

Idea:

For each number from 2 to N , check if it is divisible by any number smaller than it.

Time Complexity: $O(N^2)$

Space Complexity: $O(1)$

✗ Inefficient for large N

Optimized Approach – Sieve of Eratosthenes

Idea:

Mark multiples of each prime instead of checking divisibility.

Algorithm:

1. Create a boolean array `isPrime[]`
2. Initialize all values as `true`
3. Start marking multiples from 2
4. Stop at $\sqrt{N}$

Time Complexity: $O(n \log \log n)$

Space Complexity: $O(n)$

✓ Industry-standard solution

Java Solution (Preferred Language)

Generate All Primes Less Than N

```
import java.util.*;

class Solution {
    public List<Integer> sieveOfEratosthenes(int n) {
        boolean[] isPrime = new boolean[n];
        Arrays.fill(isPrime, true);

        isPrime[0] = false;
        isPrime[1] = false;

        for (int i = 2; i * i < n; i++) {
            if (isPrime[i]) {
                for (int j = i * i; j < n; j += i) {
                    isPrime[j] = false;
                }
            }
        }

        List<Integer> primes = new ArrayList<>();
        for (int i = 2; i < n; i++) {
            if (isPrime[i]) primes.add(i);
        }
        return primes;
    }
}
```

Bonus: Infinite Prime Generator (Without N)

Concept

Instead of generating primes up to a limit, **generate primes endlessly** using trial division against previously found primes.

Java Infinite Prime Generator

```
import java.util.*;

class PrimeGenerator {
    private List<Integer> primes = new ArrayList<>();

    public int nextPrime() {
        if (primes.isEmpty()) {
            primes.add(2);
            return 2;
        }

        int num = primes.get(primes.size() - 1) + 1;
        while (true) {
            boolean isPrime = true;
            for (int p : primes) {
                if (p * p > num) break;
                if (num % p == 0) {
                    isPrime = false;
                    break;
                }
            }
            if (isPrime) {
                primes.add(num);
                return num;
            }
            num++;
        }
    }
}
```

- ✓ Produces primes **indefinitely**
- ✓ Memory-efficient
- ✓ Common interview bonus

Practice Link (Live)

- **GeeksforGeeks**
<https://www.geeksforgeeks.org/sieve-of-eratosthenes/>
- **LeetCode**
<https://leetcode.com/problems/count-primes/>

Video Solution Link

- **YouTube**
<https://www.youtube.com/watch?v=klcIklsWzrY>

Learning Outcome

After solving this problem, the learner will be able to:

- Understand prime number generation
- Apply sieve optimization techniques
- Compare brute force vs optimized methods
- Implement infinite generators in Java
- Handle large constraints efficiently